Divide and Conquer Algorithm Design Technique

Lecture Prepared by: [Your Name]

Course: Algorithm Design and Analysis

Introduction to Divide and Conquer

- Divide and Conquer is a powerful algorithm design paradigm.
- It breaks a large problem into smaller subproblems, solves them recursively, and combines the results.

- Steps:
- 1. Divide – Split the problem.
- 2. Conquer – Solve each subproblem recursively

General Structure (Pseudocode)

- Algorithm DivideAndConquer(P):
- if (P is small enough)
- solve P directly
- else
- divide P into smaller subproblems P1, P2, ..., Pk
- for each Pi
- call DivideAndConquer(Pi)
- combine solutions of Pi to obtain solution

Common Examples

- • Merge Sort – Divide array into halves, merge sorted halves.
- • Quick Sort – Partition array around a pivot and sort subarrays.
- • Binary Search – Divide list in half and search one half.
- • Strassen’s Matrix Multiplication – Divide matrix into submatrices.

Example 1: Binary Search

- 1. Compare key with middle element.
- 2. If key < mid, search left half; else search right half.

- Time Complexity: O(log n)
- Space Complexity: O(log n) due to recursion stack.

Example 2: Merge Sort

- Steps:
- 1. Divide array into two halves.
- 2. Conquer by recursively sorting both halves.
- 3. Combine by merging sorted halves.

- Recurrence: $T(n) = 2T(n/2) + O(n)$
- $\rightarrow O(n \log n)$ by Master Theorem.

Example 3: Quick Sort

- Steps:
- 1. Choose a pivot element.
- 2. Partition array into smaller and larger elements.
- 3. Recursively sort partitions.

- Best/Average: O(n log n)
- Worst Case: $O(n^2)$ if pivot choice is poor.

Master Theorem for Analysis

- $T(n) = aT(n/b) + f(n)$

- Case 1: $f(n) = O(n^{(\log_b(a) - \varepsilon)}) \rightarrow T(n) = \Theta(n^{(\log_b(a))})$
- Case 2: $f(n) = \Theta(n^{(\log_b(a))}) \rightarrow T(n) = \Theta(n^{(\log_b(a))} \log n)$
- Case 3: $f(n) = \Omega(n^{(\log_b(a) + \varepsilon)}) \rightarrow T(n) = \Theta(f(n))$

Advantages and Disadvantages

- Advantages:
- • Simplifies complex problems.
- • Enables parallel processing.
- • Reusable recursive structure.

- Disadvantages:
- • Recursive overhead.
- • Extra memory usage.
- • Not efficient for small inputs.

Applications of Divide and Conquer

- • Sorting (Merge Sort, Quick Sort)
- • Searching (Binary Search)
- • Matrix Multiplication (Strassen’s)
- • Closest Pair of Points
- • Fast Fourier Transform (FFT)
- • Convex Hull Problems

Discussion and Review

- 1. Why is the divide step critical for performance?
- 2. Compare Divide and Conquer vs Dynamic Programming.
- 3. Derive Merge Sort time complexity using the Master Theorem.
- 4. How does recursion depth affect space complexity?